

Lab 2 — Working with Processes

Rhishikesh Tayade : CS24BT041

Himanshu Nagwanshi : CS24BT055

Part I — Single Process Search

For this part, a C++ program was written that accepts a file path, a search pattern, and a byte range as command-line arguments. The program opens the target file, moves the read pointer to the given start offset, loads the specified region into memory, and then slides a comparison window across the buffer using `memcmp` until the pattern is located.

Output:

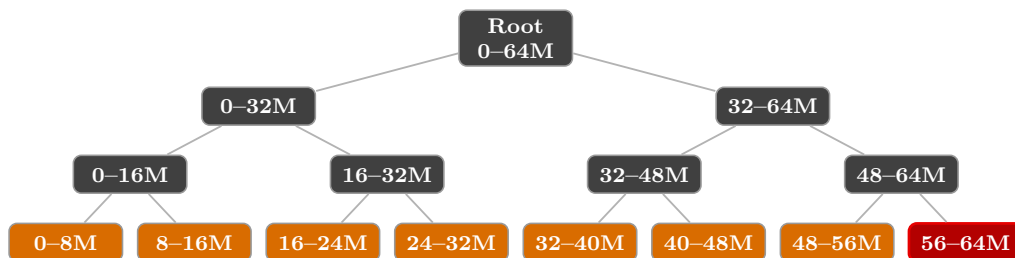
```
[5485] found at 64520807
```

The string `NGTNIJGK` turned up at byte offset `64 520 807` within the 64 MB file. While this method works correctly and is straightforward to implement, it runs entirely within a single process — meaning the full 64 MB must be scanned one step at a time, with no benefit from the machine's multiple CPU cores.

Part II — Recursive Partitioner (Multiple Processes)

To take advantage of parallel execution, a partitioner process was added to the design. The root partitioner receives the full byte range of the file along with a maximum chunk size (8 MB). Whenever its assigned range exceeds this limit, it **spawns two child partitioners** — one responsible for the left half of the range and one for the right — and then waits for both via `waitpid`. Each child applies the same splitting logic to its own half, recursively. Once a segment shrinks to 8 MB or smaller, the partitioner instead launches a single **searcher** process (using the Part I logic) and waits for it to finish.

Since every child is launched through `execv`, each node of the resulting tree runs as a fully separate process. Given a 64 MB file split into 8 MB chunks, the outcome is a three-level binary tree containing **8 leaf searcher processes, all running at the same time**.



Dark = partitioner, Orange = searcher, Red = leaf that found the pattern.

Output (excerpt):

```
[5492] start position = 0 ; end position = 67108863
[5492] forked left child 5493
[5492] forked right child 5494
...
[5514] found at 64520807
[5504] searcher child returned
```

```
...
[5492] left child returned
[5492] right child returned
```

Improvement over Part I: The workload now runs across several CPU cores at once, with the search split among 8 separate processes, each responsible for its own 8 MB slice.

Problem: The downside is that even after the pattern has already been located, the remaining searcher processes keep running and continue scanning their assigned chunks unnecessarily — wasted effort, since the answer is already known at that point.

Part III — Early Exit via SIGTERM Cascade

To fix this inefficiency, the searcher was updated so that as soon as it locates the pattern, it notifies its parent partitioner by sending `SIGTERM` through `kill(getppid(), SIGTERM)`.

Every partitioner now registers a `SIGTERM` handler before it forks any children. Upon receiving the signal, this handler:

- Logs `[pid] received SIGTERM`.
- Forwards `SIGTERM` to each of its own child processes (which it keeps track of in a list).
- Terminates right away.

The result is a **cascading shutdown**: the signal propagates upward from the searcher that found the match, through its parent, which in turn terminates its sibling branch and its own parent — continuing up the tree until every process has been shut down.

Output (excerpt):

```
[5539] found at 64520807
[5531] received SIGTERM
[5538] didn't find          <- completed before signal arrived
[5527] left child returned
...
[5521] left child returned
[5521] right child returned
```

Improvement over Part II: Any branch of the tree that hasn't finished yet gets terminated the moment the pattern is found. Branches that had already completed beforehand are unaffected — their output still shows up, but no additional work is triggered.

Efficiency gain: If the pattern happens to sit near the end of the file, Part III performs no better than Part II. But if it's located near the beginning, most searcher processes get shut down early, cutting overall CPU usage considerably.